

Full-Stack Fejlesztő

Technikai Kérdés-Válasz

Válaszgyűjtemény az SnT / Wingos interjún várhatóan felmerülő technikai kérdésekre. A cél a **tudás bizonyítása**, nem az önéletrajz felmondása.

Tartalom: **JavaScript / TypeScript · React & Next.js · Node.js · Python / FastAPI · PostgreSQL / MySQL / MongoDB · Git · Docker · CI/CD · LLM integráció · Teljesítmény · Megbízhatóság · Biztonság · Rendszer- & API-tervezés**

1 · JavaScript & TypeScript	3
2 · React & Next.js	5
3 · Node.js & Szerveroldal	8
4 · Python & FastAPI	10
5 · Adatbázisok (SQL & NoSQL)	12
6 · API-tervezés	14
7 · Git · Docker · CI/CD	15
8 · LLM integráció	17
9 · Teljesítmény · Megbízhatóság · Biztonság	18
10 · Akadálymentesség (a11y)	20

1 JavaScript & TypeScript

Nyelvi alapok, amikkel az interjúztatók gyorsan felméri a mélységet.

K Magyarázd el az event loopot. Hogyan hat egymásra a call stack, a microtask és a macrotask sor?

A JS egyszerű. A szinkron kód a **call stacken** fut. Amikor ez kiürül, a loop teljesen leüríti a **microtask sort** (Promise-callbackek, `queueMicrotask`, `await` folytatások), majd kivesz **egy** macrotaskot (`setTimeout`, I/O, UI-események), aztán újra leüríti a microtaskokat.

Fontos következmény: a microtaskok mindig a következő timer előtt futnak, így egy Promise-lánc előbb lefut, mint egy korábban beütemezett `setTimeout(..., 0)`.

Valószínű továbbkérdés: „Mi íródik ki előbb?” — egy

`Promise.resolve().then()` vs egy `setTimeout(0)`. Válasz: a Promise, mert a microtaskok a következő macrotask előtt ürülnek le.

K Mi a különbség a `==` és `===` között, és mikor elfogadható a `==`?

A `===` típust és értéket hasonlít, kényszerítés nélkül. A `==` típuskényszerítést végez. A `==` egyetlen ideomatikus használata az `x == null`, ami igaz `null`-ra és `undefined`-ra is — tömör „nincs érték” ellenőrzés. Egyébként alapból `===`.

K Closure-ök — mik ezek, és mondj rá valós példát.

A closure egy függvény, ami megtartja a hozzáférést a definíciós hatókörének változóihoz, még azután is, hogy az a hatókör visszatért. Gyakorlati példák: adatretetés (module pattern), memoizációs cache-ek, és stabil callback-állapot — pl. egy `once()` wrapper vagy egy debounce, ami a timerét a külső hatókörben tartja.

K `var` vs `let` vs `const`; magyarázd el a hoistingot és a temporal dead zone-t.

A `var` függvény-hatókörű és `undefined`-ként hoistolódik. A `let` / `const` blokk-hatókörű, hoistolódik, de inicializálatlan — a deklaráció előtti hozzáférés hibát dob (ez a **temporal dead zone**). A `const` tiltja az újraértékadást, de a hivatkozott objektum továbbra is módosítható.

K Hogyan működik a prototípusos öröklődés?

Minden objektumnak van egy belső hivatkozása (`[[Prototype]]`) egy másik objektumra. A property-keresés végigjárja ezt a láncot, amíg megtalálja, vagy `null`-ig ér. A `class` ennek szintaktikai cukorkája — a metódusok a `Prototype.prototype`-on élnek, példányok közt megosztva, nem másolva.

K Mire hivatkozik a `this`, és hogyan változtatják meg az arrow függvények?

A `this` értékét az szabja meg, **hogyan hívják** a függvényt: metódushívás → az objektum; sima hívás → `undefined` (strict) vagy global; `new` → az új példány; `call/apply/bind` → explicit. Az arrow függvényeknek **nincs saját `this`**-ük — lexikálisan a külső hatókörből öröklik, ezért ideálisak metóduson belüli callbackekhez.

K Mély vs sekély másolat — hogyan kezeled mindegyiket?

Sekély (`{...obj}`, `Object.assign`) a felső szintet másolja; a beágyazott hivatkozások közösek maradnak. Mély másolat: a `structuredClone(obj)` a modern beépített megoldás (Date-ekkel, Map-ekkel, ciklikus hivatkozásokkal is elbír). A `JSON.parse(JSON.stringify())` működik, de elhagyja a függvényeket, `undefined`-okat, és elrontja a Date-eket.

K TypeScript: `interface` vs `type`. Mikor melyiket?

- **interface** — objektum-/osztály-alakok; támogatja a declaration merginget és az `extends`-et. Nyilvános API-szerződésekhez ideomatikus.
- **type** — minden, amit az interface nem tud: union, intersection, tuple, mapped/conditional típusok, primitívek.

Ökölszabály: interface objektum-alakokhoz, type ha union vagy számított típus kell.

K Magyarázd el a generikusokat egy gyors példával.

A generikusok paraméterezik a típust, így a függvény megtartja a típusinformációt ahelyett, hogy `any`-vé omlana.

```
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}
const n = first([1, 2, 3]); // n: number
```

K Mi az `unknown`, `never` és `any` — és miért kerüljük az `any`-t?

- `any` kikapcsolja az ellenőrzést — escape hatch, ami szétviszi a típusbizonytalanságot.
- `unknown` a típusbiztos top típus: használat előtt szűkíteni kell.
- `never` a bottom típus — nincs értéke; kimerítő ellenőrzésekhez (exhaustiveness) és soha vissza nem térő függvényekhez.

K Milyen utility típusokat használsz ténylegesen?

`Partial<T>`, `Required<T>`, `Pick<T,K>`, `Omit<T,K>`, `Record<K,V>`, `Readonly<T>`, és `ReturnType<F>`. Ha párat használati esettel is meg tudsz nevezni (pl. `Omit` DTO építéséhez egy entitásból), az valós tapasztalatot jelez.

2 React & Next.js

A pozíció központi frontend keretrendszere — várható a hookok, renderelés és Next.js-specifikumok.

K Hogyan működik a React reconciliation / virtual DOM?

Állapotváltozáskor a React új virtuális fát épít, és összehasonlítja az előzővel, kiszámítva a minimális valós DOM-módosítás-halmazt. A differálás heurisztikus és $O(n)$: feltételezi, hogy eltérő elemtípusok eltérő fát adnak, és a `key`-t használja a gyerekek rendek közötti párosítására.

K Miért kell a listáknak stabil `key`, és miért csapda az `index-mint-key`?

A `key`-ek segítik a React-et az elemek rendek közötti párosításában. A tömbindex használata elromlik, ha elemeket szúrunk be / rendezünk át / törölünk — a React rossz DOM-node-ot és komponens-állapotot használ újra, finom hibákat okozva (rossz input-értékek, elcsúszott fókusz). Használj stabil domain-azonosítót.

K Magyarázd el a `useEffect`-et: dependency array, cleanup, gyakori hibák.

Render után fut. A `deps` tömb szabályozza, mikor fut újra: `[]` = egyszer mountkor; `[x]` = amikor `x` változik; kihagyva = minden renderkor. Adj vissza cleanup függvényt `subscription-ökhöz/timerekhez` — a következő effekt előtt és `unmountkor` fut.

- Hiányzó `deps` → stale closure-ök régi állapotot olvasnak.
- Objektumok/függvények a `deps`-ben → új identitás minden renderkor → végtelen ciklus.
- Effektek használata származtatott adatra, amit renderben is ki lehetne számolni.

Modern megközelítés: Mondd, hogy effekthez csak valós külső szinkronizációnál nyúlsz (fetch, subscription, DOM), nem propokból state-be alakításhoz. Pontosan ez a „You Might Not Need an Effect” irányelv.

K useMemo vs useCallback — és mikor NE használd őket?

A `useMemo` egy számított értéket cache-el; a `useCallback` egy függvényreferenciát (ez a `useMemo` függvényekre). Használd őket költséges újraszámítás elkerülésére vagy stabil referenciák megtartására memoizált gyerekekhez / effekt-depekhez. Ne szórd szét mindenhová — overheadet és bonyolultságot adnak, és a korai memoizáció gyakran többbe kerül, mint amennyit megspórol.

K Milyen problémát old meg a `useRef`, amit a `state` nem?

Módosítható konténer, ami rendereken át fennmarad, **anélkül hogy re-rendert váltana ki** módosításkor. Két használat: DOM-node-okra hivatkozás, és módosítható értékek tárolása (timer id-k, előző értékek, példányszerű flag-ek), amiknek nem szabad frissülést okozniuk.

K Rules of Hooks — mik ezek és miért?

Hookokat csak top szinten hívj (soha feltételben/ciklusban) és csak React függvényekből. A React **hívási sorrend** szerint követi a hook-állapotot, így a feltételes hívások elcsúsztatnák az állapotot renderek között.

K Hogyan kezelsz megosztott/globális állapotot anélkül, hogy azonnal Reduxhoz nyúlnál?

- **Lokális** — `useState` / `useReducer`.
- **Fán átvélő, ritkán változó** — Context (téma, auth, nyelv).
- **Szerver-állapot** — React Query / SWR (cache, revalidáció, dedup), nem kézzel írt globális store.
- **Komplex kliens-állapot** — Zustand / Redux Toolkit.

Kulcs meglátás interjún: különböztess meg a **szerver-állapotot** a **kliens-állapottól** — a legtöbb „globális állapot” valójában cache-elt szerveradat.

K Controlled vs uncontrolled komponensek?

Controlled: a React state az igazság forrása (`value` + `onChange`). Uncontrolled: a DOM tartja az értéket, ref-en át olvasva. A controlled az alapértelmezés validációhoz/dinamikus UI-hoz; az uncontrolled egyszerű vagy teljesítmény-érzékeny űrlapokhoz jó.

K Hogyan előződ meg a felesleges re-rendereket?

- `React.memo` tiszta prezentációs gyerekekhez.
- Stabil referenciák `useCallback` / `useMemo` segítségével.
- Állapot lejjebb húzása / kolokálása, hogy kevesebb komponens iratkozzon fel.
- Context felbontása vagy selectorok használata a széles invalidáció elkerülésére.

K CSR vs SSR vs SSG vs ISR Next.js-ben — kompromisszumok?

Mód	Renderelés	Mire jó
CSR	Böngészőben	Privát dashboardok, erősen interaktív
SSR	Kérésenként szerveren	Személyre szabott + SEO-érzékeny
SSG	Build időben	Főleg statikus tartalom, leggyorsabb
ISR	Statikus + háttér-revalidáció	Nagy, félig friss tartalom

Az SSR/SSG javítja az első festést és a SEO-t; a CSR a munkát a kliensre tolja.

K App Router: Server Components vs Client Components?

A **Server Componentek** (alapértelmezett) szerveren futnak, nulla JS-t szállítanak, és közvetlenül fetchelhetnek adatot / érhetik el az adatbázist. A **Client Componentek** (`"use client"`) a böngészőben futnak, és state-hez, effektekhez, eseménykezelőkhöz kellenek. A minta: az oldalak maradjanak Server Componentek, az interaktivitást told le kis Client-szeletekbe.

Wingos-releváns: Ez közvetlenül szolgálja az „intuitív felület + teljesítmény” elvárást — meg tudod fogalmazni, hogy kevesebb JS-t szállítasz az osztálytermi eszközökre, ha a fa nagy részét szerveren rendereled.

K Hogyan működik az adatlekérés / caching az App Routerben?

A `fetch` ki van egészítve caching-szemantikával — alaphoz cache-el, a revalidációt a `{ next: { revalidate: N } }` vagy a dinamikushoz a `cache: 'no-store'` kapcsolja be. A Server Componentek közvetlenül `await`-elik az adatot; a Suspense-határok streamelik a UI-t, ahogy az adat megérkezik.

3 Node.js & Szerveroldal

Backend alapok — konkurencia, async minták, és Node futtatása production környezetben.

K A Node egyszálú — hogyan kezel több ezer egyidejű kapcsolatot?

Nem blokkoló I/O plusz az event loop. A Node az I/O-t (hálózat, fájl, DNS) leadja az OS-nek / a libuv szálpooljának, és callbackeket regisztrál; a fő szál sosem blokkol várakozásra. Így egy szál sok folyamatban lévő műveletet zsonglőrködik. A CPU-igényes munka a kivétel — az blokkolja a loopot.

K Hogyan kezeled a CPU-intenzív munkát blokkolás nélkül?

- **Worker Threads** CPU-munkához, folyamaton belül megosztott memóriával.
- **Child process-ek / cluster** több mag kihasználásához.
- Kiszervezés queue + külön worker service-be nehéz job-okhoz.

K Callbackek → Promise-ok → async/await. Mit oldott meg mindegyik?

A callbackek „callback hell”-t és zavaros hibakezelést okoztak. A Promise-ok komponálhatóvá tették az aszinkron lánccal és egységes hibakezeléssel a `.catch` -en át. Az `async/await` szinkron kinézetű vezérlést ad normál `try/catch` -csel. Használj `Promise.all` -t konkurenciához, `Promise.allSettled` -t ha a részleges hiba is rendben van.

K Miben más a hibakezelés szinkron vs aszinkron esetben, és mi az unhandled rejection?

A szinkron hibák `throw` / `try-catch` -en át terjednek. Aszinkron kódban egy nem `await`-elt/nem `catch`-elt elutasított Promise **unhandled rejection**-né válik. Csomagtól az `await`-eket `try/catch` -be, használj központi hibakezelő middleware-t Express-ben, és egy elkapatlan kivételt kezelj úgy, mint jelet a logolásra és a folyamat újraindítására, ne folytasd rossz állapotban.

K Mi a middleware Express-ben, és hogyan működik a lánc?

Függvények `(req, res, next)` szignatúrával, amik sorban futnak. Mindegyik vagy válaszol, vagy meghívja a `next()` -et a vezérlés továbbadására. Logoláshoz, authhoz, body parsinghez, validációhoz. A hibakezelő middleware négy argumentumú `(err, req, res, next)`, és utoljára regisztrálódik.

K **Streamek — mik ezek és miért fontosak?**

Interfészek adat darabonkénti feldolgozásához, ahelyett hogy mindent memóriába töltenénk — readable, writable, duplex, transform. Nagy fájlkhöz/feltöltésekhez és proxyzáshoz elengedhetetlen: egy read streamet `pipe()` -olsz egy write streambe, és a memória lapos marad, mérettől függetlenül.

K **Hogyan futtatod a Node-ot megbízhatóan production-ben?**

- Több példány (cluster / konténer-replikák) load balancer mögött, minden mag kihasználásához.
- Process manager vagy orchestrator újraindít crash-nél; health checkek liveness/readiness-hez.
- Graceful shutdown: kapcsolatok elfogadásának leállítása, folyamatban lévők leürítése, DB-pool zárása `SIGTERM` -re.
- Strukturált logolás + központi konfiguráció env változókon át.

K **Hogyan tartod a titkokat és konfigot a kódon kívül?**

Env változók deploykor injektálva, secrets manager az érzékeny értékeknek, `.env` csak lokális fejlesztéshez és git-ignorálva. Sose committolj kulcsokat; validáld a kötelező env-et bootkor és bukj el gyorsan, ha hiányzik.

4 Python & FastAPI

A hirdetés Node-ot és/vagy Python/FastAPI-t fogad el — elég legyen tudni ahhoz, hogy hitelesen beszélj róla.

K Miért FastAPI Flask/Django helyett egy új API-hoz?

Natív async támogatás, automatikus kérés-/válasz-validáció **Pydantic**-kal, és automatikusan generált OpenAPI/Swagger dokumentáció a típus-annotációkból. Starlette-re (ASGI) és Pydantic-ra épül, így nagy áteresztőt és típusbiztonságot kapsz kevés boilerplate-tel.

K Mit ad a Pydantic?

Futásidejű validációt és parse-olást típus-annotált modellekből. A beérkező JSON kényszerítődik és validálódik a séma ellen; érvénytelen adat automatikusan tiszta 422-t ad. Ez a szerződés-réteg a hálózat és a Python-objektumaid között.

```
class TaskIn(BaseModel):
    title: str
    priority: int = 1

@app.post("/tasks")
async def create(task: TaskIn):
    return {"ok": True, "title": task.title}
```

K Magyarázd el az `async def`-et FastAPI-ban és a GIL-t.

Egy `async` endpoint az event loopon fut; az `await` I/O közben átadja a vezérlést, így más kérések haladhatnak. A **GIL** azt jelenti, hogy egyszerre csak egy szál futtat Python bytecode-ot, így az async I/O-igényes munkánál segít, CPU-igényesnél nem. A blokkoló hívások egy async route-ban megakasztyák a loopot — futtasd őket threadpoolban (`run_in_threadpool`) vagy használj async drivereket.

K Hogyan működik a dependency injection FastAPI-ban?

A `Depends()` rendszer kérésenként feloldja és injektálja a függőségeket (DB-session-ök, auth, beállítások), generátor-függőségeknél automatikus cleanuppal. Vékonyan tartja az endpointokat, és a megosztott dolgokat (DB-session, aktuális felhasználó) tesztelhetővé és újrafelhasználhatóvá teszi.

K Szinkron vs aszinkron DB-hozzáférés Pythonban — mi a csapda?

A nem blokkoló maradáshoz async driverek kellenek (`asyncpg` , async SQLAlchemy). Egy szinkron driver bekeverése egy async route-ba blokkolja a loopot. Ha muszáj szinkron könyvtárat használnod, futtasd threadpoolban, hogy az event loop reszponzív maradjon.

K List vs tuple vs set vs dict — gyors összevetés.

Típus	Rendezett	Módosítható	Használat
list	igen	igen	rendezett szekvencia
tuple	igen	nem	fix rekord, hashelhető kulcs
set	nem	igen	egyediség, tagság
dict	beszűrési	igen	kulcs → érték, O(1) keresés

K Mik a dekorátorok?

Egy függvény, ami egy másikat csomagol be, hogy kiterjessze a viselkedését a törzse megváltoztatása nélkül — a `@decorator` szintaxis. Routinghoz (`@app.get`), authhoz, caching-hez, időméréshez. A háttérben ez csak `fn = decorator(fn)` .

5 Adatbázisok — SQL & NoSQL

A PostgreSQL/MySQL/MongoDB mind szerepel — várható modellezés, indexelés és tranzakció-kérdés.

K SQL vs NoSQL — hogyan választasz?

Relációs (Postgres/MySQL): strukturált, kapcsolódó adat; erős konzisztencia; komplex query-k és joinok; ACID tranzakciók. **Dokumentum** (Mongo): rugalmas/fejlődő séma, denormalizált aggregátumok, horizontális skálázás, olvasás-nehéz hozzáférés egyetlen kulcs alapján. A választás a hozzáférési mintákat és konzisztencia-igényt követi, nem a hype-ot.

K Mik az ACID tulajdonságok?

Atomicity (mindent-vagy-semmit), **Consistency** (érvényes állapotból érvényes állapotba), **Isolation** (egyidejű tranzakciók nem zavarják egymást), **Durability** (committolt adat túléli a crash-t). A Postgres/MySQL(InnoDB) ezeket garantálja.

K Hogyan működnek az indexek, és mi a hátrányuk?

Általában B-tree, ami a teljes szkennelést gyors kereséssé alakítja az indexelt oszlopokon (és segíti a rendezést/tartományokat). Hátrány: extra tárhely és lassabb írás, mert minden insert/update karbantartja az indexet. Azokat az oszlopokat indexeld, amikre szűrsz/joinolsz/rendezel — ne mindent.

K Mi az N+1 query probléma, és hogyan javítod?

Egy lista lekérése (1 query), majd elemenként egy kapcsolódó sor (N query) — gyakran egy ORM/lazy loading mögött rejtve. Javítás join-nal / eager loadinggal, vagy a follow-upok batchelésével (egy **IN** query / DataLoader). Nagyon gyakori forrása a lassú endpointoknak.

K Hogyan találsz meg és javítasz egy lassú query-t?

Futtasd az **EXPLAIN ANALYZE** -t, hogy lásd a tervet és hol megy el az idő (seq scanek, rossz joinok). Adj hozzá vagy javíts indexeket, csökkentsd a kiválasztott oszlopokat, írd át a query-t, és ellenőrizd a statisztikákat. Előtte-utána mérd, ne találgass.

K Normalizáció vs denormalizáció?

A normalizáció megszünteti a redundanciát kapcsolódó táblákra bontással — kevesebb anomália, több join. A denormalizáció adatot duplikál a joinok elkerülésére olvasási sebességért, az update-bonyolultság árán. Az OLTP inkább normalizált; az olvasás-nehéz/analitika vagy dokumentumtárolók a denormalizáció felé hajlanak.

K Hogyan előzik meg a tranzakciók és izolációs szintek a konkurencia-hibákat?

Egy tranzakció atomikusan csoportosít utasításokat. Az izolációs szintek (Read Committed → Serializable) teljesítményt cserélnek védelemre a dirty read, non-repeatable read és phantom ellen. A Postgres alapból Read Committed; emeld Serializable-re, ha a konkurencia melletti helyesség számít.

K Mongo-specifikum: embedding vs referencing?

Beágyazás (embed), ha az adatot együtt érik el és korlátos (rendelés + tételsorok) — egy olvasás, nincs join. **Hivatkozás (reference)**, ha az adat megosztott, korlátlan, vagy függetlenül frissül (felhasználók ↔ posztok). A domináns query köré modellezz, és ne feledd: a dokumentumoknak 16 MB-os korlátjuk van.

6 API-tervezés

„Feature-ök építése UI, API és adatbázis között” — az API-tervezési kérdések nagyon valószínűek.

K Mitől jól tervezett egy REST API?

- Erőforrás-alapú főnevek (`/students/42/grades`), nem igék.
- HTTP metódusok a szemantikához: GET/POST/PUT/PATCH/DELETE.
- Helyes státuszkódok (200/201/204, 400/401/403/404, 409, 422, 500).
- Állapotmentes kérések; konzisztens lapozás, szűrés, verziózás.

K PUT vs PATCH vs POST, és mit jelent az idempotens?

POST létrehoz (nem idempotens). **PUT** lecseréli a teljes erőforrást (idempotens). **PATCH** részlegesen frissít. **Idempotens** = ugyanaz a kérés megismételve ugyanazt a szervertállapotot adja — a biztonságos újrapróbálkozáshoz számít. A GET/PUT/DELETE idempotens; a POST általában nem.

K REST vs GraphQL — mikor választanál GraphQL-t?

A GraphQL lehetővé teszi, hogy a kliens pontosan a szükséges mezőket kérje egy körben — jó változatos kliensekhez és mélyen beágyazott adathoz, elkerülve az over-/under-fetchinget. Ára: a caching nehezebb, és őrizni kell a query mélységét/komplexitását. A REST egyszerűbb és cache-barát az egyszerű erőforrás-hozzáféréshez.

K Hogyan kezeled az autentikációt vs autorizációt egy API-ban?

AuthN = ki vagy (hitelesítés, token/session kiadása). **AuthZ** = mit tehetsz (szerepek/jogosultságok kérésenként ellenőrizve). Gyakori minta: rövid életű JWT/ access token + refresh token, vagy szerver-session-ök; az autorizációt middleware-ben kényszerítsd, sose bízz a kliensben.

K Hogyan verziózol egy API-t és fejleszted a kliensek törése nélkül?

URL-verziózás (`/v1/...`) vagy header-alapú. Mezőket adj hozzá inkább, mint eltávolíts; az új mezők legyenek opcionálisak; deprecálj ablakkal és kommunikálj. Kerüld a válasz-alakok csendes törését.

7 Git · Docker · CI/CD

Kifejezetten elvárás. Várható gyakorlati workflow- és konténerizációs kérdés.

K Írd le a Git branching workflow-odat.

Feature branchek a `main` -ből, kicsi fókuszált commitok, PR review-val és zöld CI-vel merge előtt. Trunk-based / rövid életű branchek a merge-fájdalom csökkentésére. Legyél kész elmagyarázni a merge vs rebase-t és a konfliktus-feloldást.

K Merge vs rebase — kompromisszumok?

A **merge** megőrzi a történetet és merge commitot hoz létre — biztonságos megosztott branchekhez. A **rebase** újrátssza a commitokat egy új alapra lineáris történetért — tisztább, de átírja a történetet, ezért sose rebase-elj olyan branchet, amit mások már lehúztak. „Aranyszabály: ne rebase-elj publikus/megosztott brancheket.”

K Hogyan vonsz vissza dolgokat biztonságosan — `revert` vs `reset` ?

A `git revert` új commitot hoz létre, ami visszavon egy változást — biztonságos megosztott brancheken. A `git reset` a branch-pointert mozgatja (`--soft` stagnálten tartja a változásokat, `--hard` eldobja) — átírja a történetet, lokálisan használd. Említsd a `git reflog` -ot mint mentőhálót.

K Milyen problémát old meg a Docker, és image vs container?

Konzisztens környezetek — a „nálam működik” eltűnik, mert az app a függőségeivel együtt szállítódik. Az **image** a megváltoztathatatlan build-artefakt; a **container** annak egy futó példánya. A konténerek a host kernelét osztják, így könnyebbek a VM-eknél.

K Hogyan írsz hatékony Dockerfile-t?

- Rétegeket változási gyakoriság szerint rendezz; a `package.json`-t másold és telepítsd a függőségeket a source másolása **előtt**, hogy a deps cache-elődjön.
- **Multi-stage build**ek — teljes toolchainnel buildelj, vékony runtime image-et szállíts.
- Kis base image-ek (alpine/slim), `.dockerignore`, non-root user, rögzített verziók.

Erős jelzés: A layer caching + multi-stage buildek említése azt mutatja, hogy ténylegesen optimalizáltál image-eket, nem csak írtál egyet.

K Mit csinál egy CI/CD pipeline, és milyen szakaszokat teszel bele?

A CI automatikusan buildel és tesztel minden push-ra; a CD deploy-olja az átment buildeket. Tipikus szakaszok: install → lint/typecheck → unit/integration tesztek → build → image build & push → deploy (staging → prod). A gyors visszajelzés és a törött kód merge-elésének megakadályozása a lényeg.

K Hogyan deploy-olsz leállítás nélkül?

Rolling vagy blue-green deploy load balancer mögött, health checkek a forgalom átirányítása előtt, és a képesség az előző image-re való visszaállásra. A DB-migrációkat visszafelé kompatibilisen futtasd, hogy a régi és új verzió rövid ideig együtt élhessen.

8 LLM integráció

A Wingos „AI-alapú”, és „LLM-alapú szolgáltatások integrálását kéri, ahol valós értéket adnak”. Ez a szekció megkülönböztet.

K Hogyan integrálsz egy LLM-et egy termékbe felelősségteljesen?

Kezeld az LLM-et külső, megbízhatatlan, nem determinisztikus szolgáltatásként: validáld és korlátozd a kimenetét, kezeld a latenciát/timeoutokat/rate limiteket, streameld a válaszokat a UX-hez, és tarts fenn fallbacket. Oda tedd, ahol valóban segít (fogalmazás, magyarázat, értékelés-támogatás), ne gimmickként — tükrözve a pozíció „valós érték” keretét.

K Mi az a RAG, és mikor használod?

Retrieval-Augmented Generation: releváns dokumentumokat kérsz le (általában vektor-hasonlósági keresés embeddingeken át), és kontextusként átadod, hogy a modell a te adatodból válaszoljon, ne csak a tanítóanyagából. Használj válaszok megalapozására, források idézésére, és a hallucináció csökkentésére — pl. egy iskola saját tananyagából válaszolva.

K Mik az embeddingek és a vektoradatbázis?

Az embeddingek szöveget képeznek le magas dimenziós vektorokra, ahol a szemantikai hasonlóság $\approx$ közelség. Egy vektor-DB (pgvector, Pinecone, Qdrant) tárolja őket és gyors legközelebbi-szomszéd keresést végez. A szemantikus keresés és a RAG alapja.

K Hogyan kapsz strukturált, megbízható kimenetet egy LLM-től?

- Kérj JSON-t / használj structured-output vagy function-calling módot.
- Validáld séma ellen (Pydantic/Zod) és próbáld újra hibánál.
- Korlátozz tiszta system promptokkal és példákkal; tartsd alacsonyan a temperature-t determinisztikus feladatokhoz.

K Hogyan kezeled a költséget, latenciát és a prompt-injection kockázatot?

Cache-eld az ismétlődő hívásokat, válassz modellméretet feladatonként, korlátozd a tokeneket, és streamelj a latencia elrejtésére. Biztonsághoz: sose bízd a modell kimenetét parancsként, tisztítsd amit a promptba interpolálsz, izoláld a megbízhatatlan felhasználói tartalmat, és tartsd a modellt távol a privilegizált műveletektől validáció nélkül. Releváns a diákok általi használat miatt.

K Mi okozza a hallucinációkat, és hogyan csökkented?

A modell valószínű szöveget jósol az igazság fogalma nélkül. Enyhítsd RAG-megalapozással, forrás-idézés kérésével, alacsonyabb temperature-rel, korlátozott kimenettel, és human-in-the-loop-pal nagy tétű tartalomhoz (pl. bármi, amit diákoknak/tanároknak mutatsz).

9 Teljesítmény · Megbízhatóság · Biztonság

A pozíció kifejezetten mondja: „a platform teljesítményének, megbízhatóságának és biztonságának birtoklása.” Mindhárom várható.

K Hogyan javítod a frontend teljesítményt / Core Web Vitals-t?

- Code-splitting & lazy loading; kevesebb JS szállítása (server componentek).
- Képek optimalizálása (modern formátumok, helyes méretezés, lazy).
- Költséges munka memoizálása; hosszú listák virtualizálása.
- Statikus assetek cache-elése/CDN; layout shift (CLS) és blokkoló erőforrások csökkentése.

K Lassú a backend — hogyan diagnosztizáld és javítod?

Előbb mérj: APM/tracing a lassú span megtalálására. Szokásos gyanúsítottak: indexeletlen/N+1 query-k, hiányzó caching, szinkron blokkoló munka, csacsogó külső hívások. Javítások: indexek hozzáadása, forró olvasások cache-elése, batchelés, nehéz munka háttér-jobokba mozgatása, connection pooling.

K Hol és hogyan adsz hozzá caching-et?

Rétegek: böngésző/CDN a statikushoz, alkalmazás-cache (Redis) a forró adat-hoz/session-ökhöz, DB query cache. A nehéz rész az invalidáció — használj TTL-eket, az adathoz kötött cache-kulcsokat, és invalidálj íráskor. Azokat az olvasásokat cache-eld, amik gyakoriak és ritkán változnak.

K Nevezd meg a legfontosabb web-biztonsági kockázatokat és védelmeket.

- **SQL injection** → paraméterezett query-k / ORM, sose fűzz össze SQL-t stringből.
- **XSS** → kimenet escapelése/kódolása, input tisztítása, CSP; a React alaphól escapel (vigyázz a `dangerouslySetInnerHTML` -lel).
- **CSRF** → tokenek / SameSite sütik.
- **Törött auth** → hashelt jelszavak (bcrypt/argon2), biztonságos session/JWT kezelés.
- Least privilege, HTTPS mindenhol, függőségek naprakészen tartása.

Osztálytermi kontextus: Kiskorúak adatainak kezelése — legyél kész említeni az adatminimalizálást, hozzáférés-vezérlést és a privacy-by-design-t. Illik az SnT „Security, Reliability and Trust” fókuszához.

K Hogyan teszel egy rendszert megbízhatóvá / megfigyelhetővé?

Health checkek, strukturált logolás, metrikák és tracing (a három pillér), riasztás SLO-kra, graceful degradation, retry-k backoff-fal átmeneti hibákra, és timeoutok/circuit breakerek, hogy egy elromló függőség ne kaszkádoljon.

K Hogyan validálod és tisztítod a felhasználói inputot a stacken át?

Szerveren validálj, a kliens-ellenőrzéstől függetlenül — a kliens-validáció csak UX. Használj séma-validációt (Zod/Pydantic), engedélyezett értékek whitelistjét, kényszeríts típusokat/hosszokat, és kódolj kimeneten. Sose bízz semmiben, ami a kliensről jön.

K Hogyan tárolsz jelszavakat és kezelsz session-öket/tokeneket biztonságosan?

Hashelj lassú, szózott algoritmussal (bcrypt/argon2) — sose titkosíts vagy tárolj plaintextet. Tokenek: rövid életű access + refresh, biztonságos tárolás (a httpOnly sütik enyhítik az XSS-es token-lopást), rotálj és vonj vissza kijelentkezéskor, és mindig HTTPS-en szállíts.

10 Akadálymentesség (a11y)

A pozíció hangsúlyozza az „intuitív és akadálymentes felületeket tanároknak és diákoknak” — valószínű megkülönböztető, amit a legtöbb jelölt kihagy.

K Hogyan építész akadálymentes felületeket?

- Szemantikus HTML először (`<button>` , `<nav>` , headingek) — ez ingyen adja a billentyűzet- és képernyőolvasó-viselkedést.
- Billentyűzettel bejárhatóság és látható fókusz-állapotok.
- ARIA csak a szemantika hézagainak kitöltésére; label-ök az inputokhoz.
- Elegendő színkontraszt; ne csak színre hagyatkozz.
- Alt szöveg; tartsd tiszteletben a reduced-motion preferenciát.

K Mi a WCAG, és mit jelent a gyakorlatban az „akadálymentes”?

A WCAG az akadálymentességi szabvány, a **Perceivable, Operable, Understandable, Robust** köré szervezve, A/AA/AAA szintekkel (az AA a gyakori cél). Gyakorlatban: használható billentyűzettel, képernyőolvasóval, és gyengénlátók vagy motoros korlátozottak által. Egy diverz diákok által használt iskolai termékhez ez valódi termék-követelmény, nem pipa a listán.

K Hogyan tesztelnéd az akadálymentességet?

Az automatizált eszközök (axe, Lighthouse) elkapják az alacsonyan lógó gyümölcsöt, de a manuális tesztelés is számít: járd be az egész flow-t csak billentyűzettel, tesztelj képernyőolvasóval, ellenőrizd a kontrasztot, és verifikáld a fókusz-sorrendet. Az automata ellenőrzések a valós problémák kisebbségét fogják.